

Assignment Solutions

Part I: Short Answer / Reasoning Questions

Q1

The query returns 0 because NULL does not represent an actual value. In SQL, comparing anything with NULL using standard operators like = or != results in UNKNOWN, not TRUE. Since the WHERE clause only selects rows where the condition evaluates strictly to TRUE, no rows are returned.

The correct query is:

```
1 SELECT COUNT(*)
2 FROM Student
3 WHERE phone_number IS NULL;
```

NULL values are generally excluded from comparison and aggregate operations because SQL treats them as missing or unknown. Most aggregate functions such as AVG(), SUM(), MIN(), and MAX() ignore NULL values so that missing data does not affect or skew the calculations.

Q2

Yes, the student is correct. Since the query already groups rows by department, each department appears only once in the output. Therefore, there are no duplicate rows left for DISTINCT to remove. DISTINCT would only eliminate identical rows if duplicates existed, but GROUP BY has already ensured that each department has a single aggregated result. Hence, using DISTINCT is redundant.

Q3

An INNER JOIN only returns customers that have matching orders. A LEFT JOIN returns every customer, even those who have never placed an order. If the LEFT JOIN produces more rows than the INNER JOIN, it means that there are customers in the Customer table who do not have matching records in the Order table. These unmatched customers appear with NULL values for the order columns.

Q4

The query is invalid because employee_name is neither included in the GROUP BY clause nor aggregated using an aggregate function. When SQL groups data by department, there can be many employees in the same department, so SQL would not know which employee name to display for that department.

The correct query is:

```
1 SELECT department, AVG(salary)
2 FROM Employee
3 GROUP BY department;
```

Q5

The error is that AVG(salary) is an aggregate function, and aggregate functions cannot be used inside the WHERE clause. The WHERE clause is executed before grouping takes place, so the average salary has not yet been calculated.

The correct query is:

```
1 SELECT department, AVG(salary)
2 FROM Employee
3 GROUP BY department
4 HAVING AVG(salary) > 80000;
```

The HAVING clause filters groups after aggregation has been completed.

Q6

For part (a), a non-correlated subquery is more appropriate. The overall average product price is calculated only once and then compared with every product price.

For part (b), a correlated subquery is more appropriate. Each employee must be compared with the average salary of their own department, so the subquery depends on the department of the current employee.

In terms of performance, a correlated subquery is usually slower because it is executed once for every row of the outer query. A non-correlated subquery is evaluated only once, making it more efficient when the outer table is large.

Part II: Long Answer Questions

(a)

```
1 SELECT c.customer_id, c.name, c.email
2 FROM Customer c
3 LEFT JOIN `Order` o ON c.customer_id = o.customer_id
4 WHERE o.order_id IS NULL;
```

I chose a LEFT JOIN paired with a WHERE ... IS NULL filter because it keeps all customer records intact while safely and accurately isolating those that have no matching records in the Order table, which would be completely omitted by a standard INNER JOIN.

(b)

```
1 SELECT r.restaurant_id, r.name,
2        SUM(mi.price * oi.quantity) / COUNT(DISTINCT o.order_id) AS
3        avg_order_value
4 FROM Restaurant r
5 JOIN `Order` o ON r.restaurant_id = o.restaurant_id
6 JOIN OrderItem oi ON o.order_id = oi.order_id
7 JOIN MenuItem mi ON oi.item_id = mi.item_id
8 GROUP BY r.restaurant_id, r.name
9 HAVING COUNT(DISTINCT o.order_id) > 5;
```

I utilized a multi-table INNER JOIN to map items to orders and restaurants, calculating the total revenue divided by unique orders to find the average order value. The HAVING clause ensures we filter out any restaurants with 5 or fewer total orders.

(c)

```
1 WITH RestaurantAverages AS (
2     SELECT r.restaurant_id, r.name,
3            SUM(mi.price * oi.quantity) / COUNT(DISTINCT o.order_id) AS
4            avg_order_value
5     FROM Restaurant r
6     JOIN `Order` o ON r.restaurant_id = o.restaurant_id
7     JOIN OrderItem oi ON o.order_id = oi.order_id
8     JOIN MenuItem mi ON oi.item_id = mi.item_id
9     GROUP BY r.restaurant_id, r.name
10    HAVING COUNT(DISTINCT o.order_id) > 5
11 ),
12 PlatformAverage AS (
13     SELECT SUM(mi.price * oi.quantity) / COUNT(DISTINCT o.order_id) AS
14     global_avg
15     FROM `Order` o
16     JOIN OrderItem oi ON o.order_id = oi.order_id
17     JOIN MenuItem mi ON oi.item_id = mi.item_id
18 )
19 SELECT restaurant_id, name, avg_order_value
20 FROM RestaurantAverages
21 WHERE avg_order_value > (SELECT global_avg FROM PlatformAverage);
```

I chose to write this using a non-correlated subquery structure with common table expressions because the platform-wide average order value is a single, static value. This avoids the performance penalties associated with correlated queries.

(d)

```
1 WITH RankedItems AS (  
2     SELECT mi.restaurant_id, mi.item_id, mi.item_name, SUM(oi.quantity) AS  
3         total_qty,  
4         RANK() OVER (PARTITION BY mi.restaurant_id ORDER BY SUM(oi.quantity)  
5         DESC) AS rank_num  
6     FROM MenuItem mi  
7     JOIN OrderItem oi ON mi.item_id = oi.item_id  
8     GROUP BY mi.restaurant_id, mi.item_id, mi.item_name  
9 )  
10 SELECT restaurant_id, item_id, item_name, total_qty  
11 FROM RankedItems  
12 WHERE rank_num = 1;
```

I utilized the RANK() window function partitioned by restaurant to gracefully extract the top-performing item. A primary limitation of basic GROUP BY and MAX approaches is that while they can identify the maximum quantity, they cannot retrieve other non-aggregated details such as the item name without requiring complex self-joins.

Part III: Normalization

Q1

The functional dependencies are:

1. $\text{student_id} \rightarrow \text{student_name}$
2. $\text{course_id} \rightarrow \text{course_name}, \text{instructor_id}$
3. $\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$
4. $(\text{student_id}, \text{course_id}) \rightarrow \text{grade}, \text{enrollment_date}$

Q2

Yes. The table satisfies First Normal Form because every attribute contains a single atomic value, there are no repeating groups, and each row can be uniquely identified by the composite key (student_id, course_id).

Q3

The primary key is (student_id, course_id). However, student_name depends only on student_id, and course_name and instructor_id depend only on course_id. These are partial dependencies and violate Second Normal Form.

As an example of an anomaly, if a student's name changes, it must be updated in every row where that student is enrolled. Missing even one row would lead to inconsistent data.

Q4

The transitive dependency is $\text{course_id} \rightarrow \text{instructor_id}$ and $\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$. Therefore, $\text{course_id} \rightarrow \text{instructor_name}, \text{instructor_office}$ is a transitive dependency.

As an example of an anomaly, if an instructor changes their office, the office number must be updated in every course taught by that instructor. Failure to update all rows leads to inconsistent information.

Q5

To satisfy Third Normal Form, the single Enrollment relation is decomposed into four distinct tables:

Student(student_id, student_name)

This is justified because student_name depends only on student_id.

Instructor(instructor_id, instructor_name, instructor_office)

This is justified because instructor_name and instructor_office depend only on instructor_id.

Course(course_id, course_name, instructor_id)

This is justified because course_name and instructor_id depend only on course_id.

Enrollment(student_id, course_id, grade, enrollment_date)

This is justified because grade and enrollment_date depend on the composite key (student_id, course_id).

Part IV: Design Justification

For Part II(a), I would choose a LEFT JOIN instead of a subquery to find customers who have never placed an order. A LEFT JOIN makes the relationship between the Customer and Order tables easy to understand because it includes every customer and returns NULL for customers without matching orders. Filtering those NULL values clearly identifies customers who have never ordered. Although the same result can also be obtained using a NOT EXISTS subquery, the LEFT JOIN approach is generally easier to read and explain. It also handles NULL values naturally when checking for missing matches.

Normalizing the Enrollment table into 3NF removes redundancy and improves data consistency by storing each fact exactly once. However, it also makes queries more complex because information about students, courses, instructors, and enrollments is spread across multiple tables. Retrieving complete enrollment details now requires several JOIN operations. In real-world systems, some denormalization may be introduced to improve query performance and reduce the number of joins for frequently accessed reports, even though it increases data redundancy.